

One program, many models

The same interventions on a Petri net, a hand-written ODE, and a discrete map

A program is written once, against target names. It then applies to any model that has those targets, whatever framework built the model and whatever the time discretisation. This vignette runs one program on three representations of the same SIR model. A companion document in the `algebraicdynamics/` subdirectory, which needs Julia 1.11 for its dependencies, does the same for an `AlgebraicDynamics.jl` resource sharer.

```
using CategoricalInterventions
using AlgebraicPetri
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)

u0 = LVector(S=990.0, I=10.0, R=0.0)
p0 = LVector(inf=0.05 * 10.0 / sum(u0), rec=0.25)
tspan = (0.0, 40.0)
grid = 0.0:0.5:40.0
```

0.0:0.5:40.0

The program

```
space = TargetSpace()
declare!(space, :inf, Affine(Multiplicative())); value_type=Float64)
declare!(space, :S, Additive(); kind=State, value_type=Float64)
declare!(space, :R, Additive(); kind=State, value_type=Float64)
```

```

program = @interventions space begin
    lockdown    = scale(:inf, 0.5; during=5.0..25.0)
    vaccination = transfer(:S => :R, 50.0; at=15.0)
end

```

Program(2 atoms)

Model 1: AlgebraicPetri

```

sir = LabelledPetriNet([:S, :I, :R], :inf => ((:S, :I) => (:I, :I)), :rec => (:I => :R))
petri = Model(sir)
petri_sol = simulate(petri, program; u0, p0, tspan, alg=Tsit5(), saveat=grid)
petri_sol[3, end]

```

355.81508192595817

Model 2: a hand-written vector field

Any function that ODEProblem accepts can be a model. The targets are declared by name; labelled arrays make the names meaningful to the solver.

```

sir_ode(u, p, t) = LVector(S=-p.inf * u.S * u.I,
                           I=p.inf * u.S * u.I - p.rec * u.I,
                           R=p.rec * u.I)
direct = Model(sir_ode; parameters=[:inf, :rec], states=[:S, :I, :R],
               invariants=[Conserved([:S, :I, :R])])
direct_sol = simulate(direct, program; u0, p0, tspan, alg=Tsit5(), saveat=grid)
maximum(abs, Array(petri_sol) .- Array(direct_sol))

```

1.8250489119964186e-8

The two continuous models receive the same callback and agree to solver tolerance.

Model 3: a discrete-time map

The discrete SIR of `epirecipes/sir-julia` converts rates to per-step probabilities with $1 - \exp(-r \cdot t)$. It is a `DiscreteProblem` solved with `FunctionMap`; the model is declared `kind=:discrete`, and `dt` is passed through `simulate` to the solver.

```
rate_to_proportion(r, t) = 1 - exp(-r * t)
function sir_map!(du, u, p, t)
    infection = rate_to_proportion(p.inf * u.I / p.N, p.t) * u.S
    recovery = rate_to_proportion(p.rec, p.t) * u.I
    du.S = u.S - infection
    du.I = u.I + infection - recovery
    du.R = u.R + recovery
    return nothing
end
t = 0.1
p0d = LVector{Float64}(inf=0.05 * 10.0, rec=0.25, N=sum(u0), t=t)
discrete = Model{Float64}(sir_map!; parameters=[:inf, :rec, :N, :t], states=[:S, :I, :R], kind=:discrete, dt=t)
discrete_sol = simulate(discrete, program; u0, p0=p0d, tspan, alg=FunctionMap(), dt=t, saveat=0.1)
discrete_sol[3, end], sum(discrete_sol.u[end])
```

(363.76340563752046, 1000.0000000000005)

The discrete model is not expected to match the ODE numerically. What is guaranteed is that its *parameter schedule* is the program's schedule sampled on the grid: applying the program at grid times equals applying the grid-sampled program at grid indices.

```
steps = collect(0.0:t:40.0)
baseline = Dict{Symbol, Float64}(:inf => fill(p0d.inf, length(steps)))
direct_schedule = apply(program, baseline, steps)[:inf]
sampled_program = sample(program, 0.0:t:40.0)
sampled_schedule = apply(sampled_program, baseline, collect(1:length(steps)))[:inf]
direct_schedule == sampled_schedule, sampled_program.atoms
```

(true, Atom[lockdown: Parameter:inf scale(0.5) on [51, 251), vaccination: State:S transfer(5

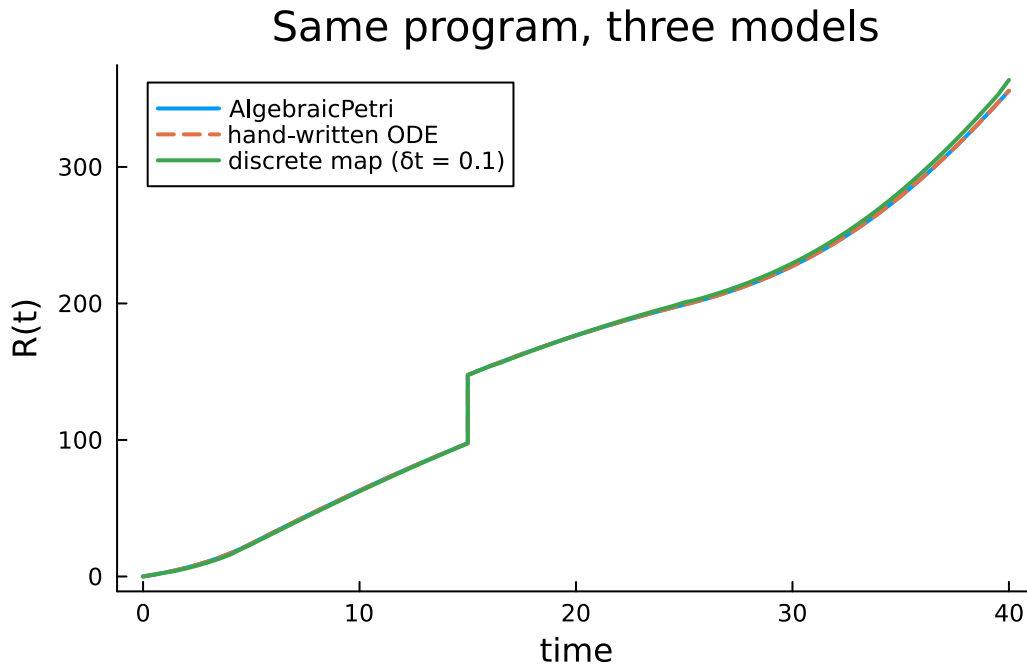
The event table the callback executes is the same schedule again.

```
table = lower(program, Dict{Symbol, Float64}(:inf => p0d.inf))
all(hold_last(table, Dict{Symbol, Float64}(:inf => p0d.inf), t)[:inf] == direct_schedule[i] for (i, t) in enumerate(steps))

true
```

Comparison

```
plot(petri_sol.t, petri_sol[3, :]; label="AlgebraicPetri", xlabel="time", ylabel="R(t)",  
     title="Same program, three models")  
plot!(direct_sol.t, direct_sol[3, :]; label="hand-written ODE", linestyle=:dash)  
plot!(discrete_sol.t, discrete_sol[3, :]; label="discrete map ( t = 0.1)")
```



What the category theory bought

Applying a program commutes with resampling the timeline: pulling the program back along the grid map and applying at indices equals applying at grid times (`apply_pullback`, `grid_preimage_span`, `apply_discrete_eq`). The callback executes exactly the schedule the program defines (`lower_eq_apply`). Framework independence is therefore not a coincidence checked case by case: the program's meaning is fixed before any model is chosen, and each model only supplies selectors for the target names.